

Regression & Classification Models

From Tanzania tourism spending to Kenyan loan defaults, turn preprocessed data into predictions.

By end of this week, you will be able to

Explain the difference between regression and classification tasks

Split a dataset into training and test sets correctly

Train and evaluate linear and polynomial regression models

Train logistic regression, decision tree, and KNN classifiers

Interpret regression metrics: MAE, MSE, RMSE, and R^2

Interpret classification metrics: accuracy, precision, recall, F1, and confusion matrix

Compare multiple models on the same dataset and justify a recommendation

1. From Data to Decisions

Apollo Agriculture uses satellite and historical yield data to predict two things for smallholder farmers in Nakuru: whether a farmer will repay a loan (a yes/no question) and how many bags of maize their farm will produce (a number). The first is classification, the second regression.

Task	Output	Key metrics	Kenyan example
Regression	A number	RMSE, R^2	Predict tourist spending USD
Classification	A category	F1, Precision	Predict loan default Yes/No

Both use the same workflow: split → preprocess → train → evaluate. You already have preprocessed DataFrames from Week 3 and will plug them directly into models today.

2. The Train/Test Split

You cannot evaluate a model on the data it learned from. Imagine a teacher who gives students the exact exam questions to practise. Every student scores

100% but no one learns anything. A model that memorises training data does the same thing. That is overfitting.

Standard split: 80% train, 20% test. Use `random_state` for reproducibility. Validation sets help with hyperparameter tuning, which we cover more in Week 8.

Lab 1 – Train/test split • [Run in Colab](#)

```
import pandas as pd
from sklearn.model_selection import train_test_split

# Use your preprocessed Tanzania Tourism DataFrame from Week 3
df_tourism = pd.read_csv("tanzania_tourism_preprocessed.csv")

X = df_tourism.drop("total_cost", axis=1)
y = df_tourism["total_cost"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Training set: {X_train.shape[0]} rows")

print(f"Test set: {X_test.shape[0]} rows")
```

Always split BEFORE fitting any preprocessor. Reference the data leakage section from Week 3.

3. Linear Regression

The equation: $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$. The model finds coefficients that minimise the sum of squared errors between predictions and actual values.

Think of predicting Nairobi house prices from square footage, bedrooms, and distance from the CBD. The model learns how each feature contributes to the final price.

Lab 2 – Linear regression • Run in Colab

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²: {r2:.4f}")

# Visualise actual vs predicted
plt.figure(figsize=(7, 5))
plt.scatter(y_test, y_pred, alpha=0.5, color="#1D9E75")
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()], "r--", lw=2)
plt.xlabel("Actual Spending (USD)")
plt.ylabel("Predicted Spending (USD)")
plt.title("Linear Regression: Actual vs Predicted")
plt.tight_layout()
plt.show()

# Residuals
residuals = y_test - y_pred
residuals.hist(bins=30, color="#EF9F27", edgecolor="white")
```

```
plt.title("Residuals Distribution")
plt.xlabel("Prediction Error (USD)")
plt.tight_layout()
plt.show()

# Coefficients
coef_df = pd.DataFrame({
    "feature": X_train.columns,
    "coefficient": lr.coef_
}).sort_values("coefficient", ascending=False)

print(coef_df.to_string(index=False))
```

RMSE is in the same units as the target (USD). "On average our prediction is off by X USD." R^2 tells you the proportion of variance explained.

Polynomial Regression

Use when the relationship curves. For example, age vs income peaks mid-career.

Lab 3 – Polynomial regression • [Run in Colab](#)

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

lr_poly = LinearRegression().fit(X_train_poly, y_train)

print(f"Polynomial R²: {r2_score(y_test, lr_poly.predict(X_test_poly)):.4f}")
```

Plot your residuals. If they fan out as predictions grow (heteroscedasticity), your model needs log-transforming the target or switching to a tree-based model.

4. Regression Metrics

MAE (Mean Absolute Error)

Average of $|\text{actual} - \text{predicted}|$

Same units as target, easy to interpret

Robust to outliers

Use when all errors are equally important

MSE (Mean Squared Error)

Average of $(\text{actual} - \text{predicted})^2$

Penalises large errors heavily

Not in original units (USD²)

Used during model optimisation

RMSE (Root Mean Squared Error)

$\sqrt{\text{MSE}}$ — returns to original units

Most commonly reported regression metric

Sensitive to large errors

Best when big errors are costly

R² (Coefficient of Determination)

Range: 0 to 1 (can be negative for terrible models)

0.85 = model explains 85% of variance in target

Context-dependent: $R^2=0.6$ is good in social science, poor in physics

Does not confirm whether predictions are useful in absolute terms

A high R^2 does not mean your model is correct. A scatter plot of actual vs predicted tells you more than a single number. A perfect diagonal line means a perfect model. A cloud means your features are missing something.

5. Logistic Regression

Despite the name, logistic regression is a classifier. It predicts the probability that an observation belongs to a class, then applies a threshold (default 0.5) to make the binary call.

Equation: $P(y=1) = 1 / (1 + e^{(-z)})$ where $z = \beta_0 + \beta_1 x_1 + \dots$. The sigmoid function squashes any number into (0, 1).

Equity Bank's fraud detection model takes transaction features (amount, time, location, recipient type) and outputs $P(\text{fraud}=1)$. If probability > 0.3, flag for review. Lowering the threshold catches more fraud but also generates more false alarms. That is the precision/recall tradeoff.

Lab 4 – Logistic regression • Run in Colab

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix)

# Use preprocessed Loan Default dataset
clf_lr = LogisticRegression(max_iter=1000, random_state=42)
clf_lr.fit(X_train_clf, y_train_clf)
y_pred_lr = clf_lr.predict(X_test_clf)

print(f"Accuracy: {accuracy_score(y_test_clf, y_pred_lr):.4f}")
print(classification_report(y_test_clf, y_pred_lr,
                             target_names=["No Default", "Default"]))

# Confusion matrix heatmap
cm = confusion_matrix(y_test_clf, y_pred_lr)
sns.heatmap(cm, annot=True, fmt="d", cmap="Greens",
            xticklabels=["No Default", "Default"],
            yticklabels=["No Default", "Default"])
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.title("Confusion Matrix – Logistic Regression")
plt.tight_layout()
plt.show()

# Probability scores (not just hard labels)
y_proba = clf_lr.predict_proba(X_test_clf)[:, 1]

print(f"Sample probabilities: {y_proba[:5].round(3)}")
```

6. Classification Metrics

Start with the confusion matrix. Four cells explained with Loan Default context:

- **TP:** predicted Default, actually Default → correct catch
- **TN:** predicted No Default, actually No Default → correct clearance
- **FP:** predicted Default, actually No Default → wrongly blocked customer
- **FN:** predicted No Default, actually Default → missed bad loan

Metric	Formula	Prioritise when...
Accuracy	$(TP + TN) / N$	Classes are balanced

Precision	$TP / (TP + FP)$	False alarms are costly
Recall	$TP / (TP + FN)$	Missing a positive is costly
F1	$2 \times (Precision \times Recall) / (Precision + Recall)$	Imbalanced classes, need balance

Accuracy is useless for imbalanced classes. If 95% of loans don't default, a model that always predicts "No Default" achieves 95% accuracy and catches zero defaulters.

Lowering the decision threshold ($0.5 \rightarrow 0.3$) raises recall (catch more defaults) but lowers precision (more false alarms).

7. Decision Trees

Decision trees are flowcharts of yes/no questions that split data into increasingly pure groups. Each split chooses the feature and threshold that best separates classes using Gini impurity or information gain.

NHIF eligibility screening uses similar logic: employment status → income bracket → age → Eligible/Not Eligible. The tree mirrors how a human officer would ask questions, but at scale and consistently.

Lab 5 – Decision tree classifier • [Run in Colab](#)

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report

dt = DecisionTreeClassifier(max_depth=4, random_state=42)
dt.fit(X_train_clf, y_train_clf)

print(f"Accuracy: {accuracy_score(y_test_clf, dt.predict(X_test_clf)):.4f}")
print(classification_report(y_test_clf, dt.predict(X_test_clf),
    target_names=["No Default", "Default"]))

# Visualise the tree
fig, ax = plt.subplots(figsize=(16, 6))
plot_tree(dt, feature_names=X_train_clf.columns,
    class_names=["No Default", "Default"],
    filled=True, ax=ax, fontsize=8)
plt.title("Decision Tree – Loan Default (max depth=4)")
plt.tight_layout()
plt.show()

# Feature importances
importances = pd.Series(dt.feature_importances_,
    index=X_train_clf.columns)
importances.sort_values().tail(10).plot(
    kind="barh", color="#1D9E75", figsize=(8, 5))
```

```
plt.title("Top 10 Feature Importances")
plt.tight_layout()
```

```
plt.show()
```

A tree with no depth limit achieves near-100% training accuracy and often terrible test accuracy. It has memorised the training data, including noise. Always set `max_depth`, `min_samples_split`, or `min_samples_leaf`.

8. K-Nearest Neighbours

To classify a new point, find its K most similar training examples and take a majority vote. "Similar" means closest by Euclidean distance.

For a new M-Pesa agent in Kiambu, you can predict expected monthly transaction volume by averaging the volumes of the K most similar existing agents (by location, population density, and area income level).

Lab 6 – KNN classifier • [Run in Colab](#)

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

k_scores = {}
for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
```

```

knn.fit(X_train_clf, y_train_clf)
k_scores[k] = accuracy_score(y_test_clf, knn.predict(X_test_clf))

best_k = max(k_scores, key=k_scores.get)
print(f"Best K: {best_k}, Accuracy: {k_scores[best_k]:.4f}")

plt.figure(figsize=(10, 4))
plt.plot(list(k_scores.keys()), list(k_scores.values()),
         marker="o", color="#1D9E75")
plt.axvline(best_k, color="#D85A30", linestyle="--",
            label=f"Best K = {best_k}")
plt.xlabel("K (number of neighbours)")
plt.ylabel("Test Accuracy")
plt.title("KNN: Test Accuracy vs K")
plt.legend()
plt.tight_layout()

plt.show()

```

K=1 memorises training data — overfitting. Very large K is too blunt — underfitting. The elbow in the accuracy curve shows the sweet spot. Always scale your features before KNN. Distance is meaningless when one feature is in Ksh millions and another is a 0/1 flag.

9. Comparing Models Side by Side

```

Lab 7 — Model comparison • Run in Colab

import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000,
random_state=42),

```

```

    "Decision Tree (d=4)": DecisionTreeClassifier(max_depth=4,
random_state=42),
    f"KNN (k={best_k})": KNeighborsClassifier(n_neighbors=best_k)
}

results = []
for name, model in models.items():
    model.fit(X_train_clf, y_train_clf)
    preds = model.predict(X_test_clf)
    results.append({
        "Model": name,
        "Accuracy": round(accuracy_score(y_test_clf, preds), 4),
        "F1 Score": round(f1_score(y_test_clf, preds, average="weighted"), 4)
    })

print(pd.DataFrame(results).sort_values("F1 Score", ascending=False)

.to_string(index=False))

```

No model wins universally. Decision trees are interpretable — you can explain the decision to a loan applicant under Kenya's data protection requirements. Logistic regression outputs probabilities useful for risk scoring. KNN needs no training time but slows down as the dataset grows. The right choice depends on interpretability requirements, dataset size, and how costly each error type is.

For your capstone, always compare at least two models. Report accuracy AND F1. Include a confusion matrix — it tells a richer story than a single number ever can.

10. Overfitting, Underfitting & Cross-Validation

The bias-variance tradeoff defines three scenarios:

- **Underfitting (high bias):** model too simple, high error on both train and test.
- **Good fit:** low train error, low test error. Generalises to new data.
- **Overfitting (high variance):** low train error, high test error. Memorised noise.

Lab 8 – Cross-validation • Run in Colab

```
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
```

```
cv_scores = cross_val_score(
    DecisionTreeClassifier(max_depth=4, random_state=42),
    X_clf, y_clf, cv=5, scoring="f1_weighted"
)
print(f"CV F1 scores: {cv_scores.round(4)}")
```

```
print(f"Mean: {cv_scores.mean():.4f} Std: {cv_scores.std():.4f}")
```

CV splits data into 5 folds, trains on 4, tests on 1, rotates. Mean score is more reliable than a single split. Low std means a stable model. High std means performance depends heavily on which samples the model sees — a sign of high variance.

To Vaccinate or Not to Vaccinate — Sentiment Analysis

Classify African Twitter posts about COVID-19 vaccines. Use numeric/categorical features this week; NLP with raw text comes in Week 8.

[Open Zindi Challenge](#)

Resources

Official Documentation

LinearRegression

Scikit-learn

Free

LogisticRegression

Scikit-learn

Free

DecisionTreeClassifier

Scikit-learn

Free

Model Evaluation Guide

Scikit-learn

Free

Interactive Courses

Intro to Machine Learning

Kaggle Learn

Free

Intermediate Machine Learning

Kaggle Learn

Free

Supervised Learning with Scikit-learn

DataCamp

Free via Ngao

Books & Deep Dives

ISLR Chapters 3–4: Linear & Logistic Regression

Free PDF

Free

Hands-On ML Chapter 3–6

Aurélien Géron, Free Notebooks

Free

Python DS Handbook 5.02

Introducing Scikit-learn

Free

Africa Datasets

Tanzania Tourism Prediction

Zindi, Regression this week

Africa

Loan Default Prediction Nigeria

Zindi, Classification this week

Africa

To Vaccinate or Not

Zindi, This week's challenge

Africa

Academic References

1. James, G. et al. (2023). An Introduction to Statistical Learning. Springer. Free PDF at statlearning.com
2. Breiman, L. et al. (1984). Classification and Regression Trees. Chapman & Hall. — The original CART/decision tree paper.
3. Cover, T. & Hart, P. (1967). Nearest neighbor pattern classification. IEEE Trans. Information Theory, 13(1), 21–27. doi.org/10.1109/TIT.1967.1053964
4. Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861–874. doi.org/10.1016/j.patrec.2005.10.010
5. Pedregosa, F. et al. (2011). Scikit-learn: ML in Python. JMLR, 12, 2825–2830. jmlr.org/papers/v12/pedregosa11a.html